Predicting the Time Required to Find Street Parking in New York City Remains Difficult to Model

Deleted: Cannot Be Modeled Very Well

Introduction

Anyone who has driven in New York City understands that finding street parking is not just a small inconvenience - it can completely change how a trip goes. Sometimes you find a spot almost immediately, and other times where you spend ten, fifteen, or even twenty minutes circling the same few blocks. This inconsistency is what makes the experience frustrating, but also what makes it interesting from a modeling perspective.

At first glance, it feels like there should be a pattern. Certain neighborhoods are known to be worse than others, certain times of day are clearly more crowded, and weekends behave differently than weekdays. Because of this, it seems reasonable to believe that we should be able to predict how long it takes to find parking using data. If such a model existed and worked well, it could be extremely useful. Navigation systems could incorporate expected parking time into routes, drivers could make better decisions about whether to use garages, and city planners could identify areas where parking demand exceeds supply.

However, once you think about the process more carefully, it becomes clear that there are many elements involved that are either difficult or impossible to observe. Whether or not a driver finds parking quickly can depend on small, unpredictable events, like whether someone happens to leave a spot at the right moment. These types of factors introduce randomness that may limit how well any model can perform.

This raises the central question of this paper: can the time required to find street parking in New York City be modeled accurately, or is the phenomenon inherently too unpredictable? While it is possible to construct a model using observable features, I argue that the model will not perform very well due to large amounts of error, particularly error due to ignorance of true causal drivers.

Attempts to address urban parking availability are not new. Google Maps introduced a parking difficulty feature in 2017, available in New York City and other major metros, using historical data to estimate whether parking will be easy, medium, or limited at a destination (Google, 2017). New York City's Department of City Planning has conducted multiple studies examining parking supply and demand in Manhattan, documenting the challenge of balancing availability against congestion and environmental concerns (NYC Department of City Planning, 2009; 2011). San Francisco's SFpark pilot program, which ran from 2011 to 2013, used sensor-based technology and demand-responsive pricing to help drivers locate open spots in real time (SFMTA, 2013). Despite these efforts, none have solved the problem of predicting how long an individual driver will spend searching for street parking - the feature does not predict search time, only general difficulty, and the city programs focus on supply management rather than individual prediction. The problem remains resistant to accurate modeling due to the unpredictability of spot turnover and competing driver behavior.

Definition of the Phenomenon

A *phenomenon* is something occurring in the real world that we are interested in understanding or predicting. In this case, the phenomenon is the time required for a driver to find a legal street parking space in New York City.

Formatted: Font: Italic, Complex Script Font: Italic

To model this phenomenon, it must be represented using a measurable quantity. Let

$$y = \textit{time required to find parking (in minutes)}.$$

This is a univariate, continuous response variable where

$$y \in Y = [0, \infty)$$

since time cannot be negative. This measurement begins when a driver starts actively searching for parking and ends when the driver successfully parks.

The measurement of y will be obtained using driver-reported timestamps. Each driver records the time at which they begin searching and the time at which they park, and the difference gives the value of y. While this method is not perfectly precise, it is practical and allows us to collect many observations.

One potential source of bias is that drivers may inconsistently define when their search begins. For instance, some may start the clock when they exit the highway, while others may wait until they are actively circling a block. Standardizing this definition across observations would improve consistency.

Definition of a Model and Mathematical Model

A model approximates a real-world phenomenon that allows us to describe or predict it. In this case, we seek to construct a model that predicts parking search time based on observable characteristics of the environment.

A mathematical model expresses this relationship numerically. Ideally, the phenomenon is generated by some unknown true function

$$y = t(z_1, z_2, \ldots, z_t)$$

Formatted: Centered

Formatted: Centered

Deleted: y

Deleted: -

Deleted: f

Formatted: Centered

where $z_1, ..., z_t$ are the true causal drivers. These are the real factors that determine how long it takes to find parking.

However, these causal drivers are not observable in practice, so we instead construct an approximation

$$y = f(x_1, x_2, ..., x_p) + \delta$$

where:

- $x_1, ..., x_p$ are observable features
- f is an unknown function relating features to the response
- $\delta = t - f$ represents error due to ignorance

The goal of modeling is to estimate a function g that approximates f.

Formatted: Complex Script Font: Italic

Formatted: Complex Script Font: Italic

Formatted: Centered

Model Target and Users

The model target in this context is the expected parking search time y. This is a regression problem - we are directly predicting a numeric outcome rather than a category.

The primary users of this model would be drivers and navigation systems. Drivers would use the model to decide whether to search for street parking or choose an alternative, while navigation systems could incorporate parking time into route optimization.

True Causal Drivers

Before constructing any model, it is important to understand what actually generates the phenomenon in reality. In theory, the time required to find parking is determined by a set of true causal drivers, which we denote as

$$y = t(z_1, z_2, \ldots, z_t)$$

Formatted: Centered

These variables $z_1, \ldots, z_t$ represent the real underlying factors that directly determine how long it takes to find parking. Some possible causal drivers in this context include:

- the exact number of available parking spaces at a given moment
- the number of other drivers searching for parking in the same area
- the timing of when parked vehicles leave
- driver behavior, such as how aggressively or efficiently someone searches
- enforcement activity, such as ticketing or towing

While these are clearly important, there is a major issue. These causal drivers are either impossible to measure ahead of time or inherently unpredictable. For example, there is no way to know exactly when a parked car will leave or how many other drivers will arrive in the next few minutes.

Because of this, it is not possible to construct a model directly using the z's. This leads to the need for proxy variables.

One thing that stood out when thinking through the causal drivers is how sensitive this problem is to timing. Even if all the observable conditions are exactly the same, two drivers could have completely different outcomes just based on when they arrive. If one driver arrives right after someone leaves a parking spot, they might find parking immediately, while another driver arriving just a minute earlier might have to search for much longer. This kind of sensitivity to timing makes the phenomenon inherently unstable.

Features as Proxies

Since the true causal drivers cannot be observed, we instead use features $x_1, x_2, \ldots, x_p$ that act as proxies for the unknown z's. These features capture partial information about the system and allow us to approximate the true function t.

Thus, we move from

$$y = t(z_1, z_2, \ldots, z_t)$$

Formatted: Centered

to

$$y = f(x_1, x_2, \ldots, x_p) + \delta$$

Formatted: Centered

where δ represents the information we lose by not knowing the true causal drivers.

In this model, the features are chosen based on what can realistically be observed and measured by a driver or through simple data collection.

Description of Features

The features used in this model are selected to approximate the underlying causes of parking availability. Let p denote the number of features.

Deleted: p

We define the following features:

- x_1: neighborhood or location
- x_2: time of day (measured in hours, e.g. 0–23)
- x_3: day of the week (encoded categorically)
- x_4: perceived traffic level (low, medium, high, converted to numeric scale)
- x_5: number of parking restrictions in the area (such as street cleaning schedules)
- x_6: weather conditions (e.g. clear, rain, snow, encoded numerically)

- x_7: proximity to high-demand areas (distance to restaurants, offices, etc.)

So, the total number of features is $p = 7$.

Of these, x_1 and x_3 are categorical variables that require encoding (for example, one-hot encoding for neighborhood), while x_2, x_4, x_5, and x_7 are treated as continuous, and x_6 is ordinal.

Each of these features is measurable in practice. For example, time of day can be recorded precisely, weather conditions are publicly available, and parking restrictions can be determined from posted signs. While some features, like traffic level, are more subjective, they can still be reasonably approximated by drivers.

These features form the design matrix $X \in \mathbb{R}^{n \times (p+1)}$, where each row vector $x_{1\cdot}, x_{2\cdot}, \ldots, x_{n\cdot}$ represents one observation and each column vector $x_{\cdot 1}, x_{\cdot 2}, \ldots, x_{\cdot p}$ represents one feature across all observations.

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$

Training Data

To construct the model, we need historical observations of parking searches which is known as the training data. These observations form the dataset

$$D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

where n is the number of observations.

Formatted: Centered

Each observation corresponds to one instance of a driver searching for parking. For example, one row in the dataset might look like:

- location: Manhattan
- time: 7 PM
- day: weekday
- traffic: high
- restrictions: active street cleaning
- weather: rain
- distance to restaurants: close
- *y = 18 minutes*

$$x_i = \begin{pmatrix} Manhattan \\ 19 \\ weekday \\ high \\ active \\ rain \\ close \end{pmatrix}, y_i = 18$$

Formatted: No bullets or numbering

In practice, this data would be collected using driver-reported timestamps. Each driver records when they begin searching and when they park, and the difference gives the value of *y*.

Feasibility of Data Collection

One important question is whether it is realistic to collect a large dataset of this type. In this case, the answer is yes.

Each parking attempt provides one observation, and since parking is a very common activity in New York City, it is possible to collect a large number of observations. If multiple

drivers contribute data over time, the dataset could grow quickly. So, we can assume that n is large, which is beneficial for model estimation.

Data could most realistically be collected in one of two ways. A standalone app with a simple start and stop button, allowing drivers to manually log their searches, which could potentially be incentivized through some sort of rewards system or "gamification". Another method would be to build functionality into existing navigation apps, such as Waze or Google Maps, that would automatically log timestamps without requiring manual input. This could be done through a GPS based approach, either detecting when a car starts moving slowly through a small area as a proxy for searching behavior, or by triggering the start of search when the driver's destination has been reached. While this would decrease data loss because of parkers who fail to log their search, the passive collection also increases measurement error, leaving a tradeoff between data completeness and measurement precision.

Contributors would be NYC residents or frequent visitors such as commuters and delivery drivers, since they accumulate many observations over time and behave consistently. A sufficiently large n is needed to get stable estimates across a range of feature combinations, and given how many neighborhoods, times of day, and weather conditions exist in NYC, that number would likely be in the thousands.

One potential concern with self-reported data is selection bias - drivers who struggle to find parking for an unusually long time may be more or less likely to report it, which could skew the distribution of y in the dataset.

Formatted: Indent: First line: 0.5"

Deleted: ¶

Formatted: Indent: First line: 0"

Formatted: Font: Italic, Complex Script Font: Italic

Supervised Learning

Since we have labeled data, this problem falls under supervised learning. Supervised learning is the process of using historical data to learn a function that maps inputs to outputs. In this framework, we define:

- *D*: training data
- *H*: candidate set of functions
- *A*: algorithm

The model is then given by

$g = A(D, H)$

This means that the algorithm takes in the data and searches through the set of possible functions to find the best approximation.

Machine Learning Perspective

Machine learning refers to the broader framework of using algorithms to learn patterns from data. In this context, we are using machine learning to estimate the function f that relates observable features like location, time, and day to parking time. Rather than manually specifying how each of these features combine to produce a prediction, machine learning lets the data determine those relationships automatically by finding the function in *H* that best fits the training observations.

The goal is not to perfectly recover the true function t, which is impossible given that the true causal drivers are unobservable. Instead, it is to construct a model *g* that performs well in

Formatted: Font: Italic, Complex Script Font: Italic

Deleted: ,

Deleted: but

Formatted: Font: Italic, Complex Script Font: Italic

predicting future observations, generalizing accurately to new observations. This means it captures actual patterns in the data rather than memorizing noise specific to the training set.

Prediction Using the Model

Once the model g is constructed, it can be used to make predictions. Given a new input x^* describing current conditions – such as neighborhood, weather, etc. -, the model outputs $\hat{y} = g(x^*)$

This predicted value represents the estimated time required to find parking under those conditions. The model does not need to know anything about the true causal drivers to produce this estimate; it only uses the observable features captured in x^*. This is both a strength and a limitation. Predictions can be made easily, but they will always have some uncertainty due to the unobservable factors that actually drive the outcome.

Candidate Set of Functions H

At this point, we have defined the phenomenon, selected features, and collected training data. However, we still face an important problem: how do we actually combine the features to produce a prediction?

In theory, the function $f(x_1, x_2, \ldots, x_p)$ that relates the features to the response could be arbitrarily complex. It could include nonlinear relationships, interactions, and unknown structures. Because of this, the space of all possible functions is far too large to search through.

To make the problem manageable, we restrict ourselves to a smaller set of possible functions, called the candidate set H.

A natural starting point is to assume a linear relationship between the features and the response. This leads to the candidate set

$$H = \{w_0 + w_1x_1 + w_2x_2 + \ldots + w_px_p : w_j \in \mathbb{R}\}$$

This set represents all linear functions of the features. While this is a simplification, it allows us to construct a model that is interpretable and computationally feasible.

We assume that there exists a best function within this set, denoted

$$h^* \in H$$

which is the best possible approximation of the true function f within the restricted space H.

Algorithm A

Once we have defined the candidate set H, we need a method for selecting the best function from this set using the training data. An algorithm, denoted A, is a procedure that takes in the training data D and the candidate set H, and selects a function g from within H.

In this context, we use Ordinary Least Squares (*OLS*) as the algorithm. *OLS* selects the function that minimizes the sum of squared errors between the observed values and the predicted values:

$$SSE = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

The result of this process is a vector of coefficients $\hat{b}$, which defines the model

$$g(x) = \hat{b}_0 + \hat{b}_1x_1 + \ldots + \hat{b}_px_p$$

So, the full modeling process can be written as

$$g = A(D, H)$$

where g is the final model used for prediction.

Formatted: Centered
Formatted: Centered
Formatted: Font: Italic, Complex Script Font: Italic
Deleted: This method is called the algorithm, denoted *A*.
Formatted: Font: Not Italic, Complex Script Font: Not Italic
Formatted: Font: Not Italic, Complex Script Font: Not Italic
Formatted: Font: Not Italic, Complex Script Font: Not Italic
Formatted: Font: Not Italic, Complex Script Font: Not Italic
Formatted: Font: Not Italic, Complex Script Font: Not Italic
Formatted: Font: Italic, Complex Script Font: Italic
Formatted: Font: Italic, Complex Script Font: Italic
Formatted: Centered
Formatted: Centered
Formatted: Centered

Null Model g_0

Before evaluating the performance of our model, it is important to establish a baseline. This baseline is known as the null model, denoted g_0.

The null model ignores all features and simply predicts the average value of the response:

$$g_0(x) = \bar{y}$$

Formatted: Centered

where $\bar{y}$ is the mean parking time in the dataset.

For example, if the average parking search time in the dataset turns out to be around 10 minutes, then any useful model must predict more accurately than that flat estimate.

Any useful model should perform better than the null model. If our model does not improve upon this baseline, then the features are not providing meaningful predictive information.

Three Sources of Error

Even with a well-defined model, there will always be error in predictions. There are three main sources of error in this framework.

1. Error Due to Ignorance

This type of error arises because we do not know the true causal drivers $z_1, \ldots, z_t$. Since we use proxy variables instead, we lose information about the system.

Mathematically, this error is represented as

$$\delta = t(z) - f(x)$$

Formatted: Centered

In the context of parking, this error is likely to be very large. Many important factors, such as the exact timing of when a parking spot becomes available, are inherently unpredictable. No matter how many features we include, we cannot fully capture these effects.

What I found interesting here is that this type of error is not something we can really fix by collecting more data. Normally, if a model is not performing well, the first instinct is to increase the sample size or improve feature quality. However, in this case, the main issue is that some of the most important variables are fundamentally unobservable. This means that no matter how large n becomes, this source of error will still remain.

Because of this, it changes how I think about the goal of modeling in this context. Instead of trying to achieve very high accuracy, the goal becomes understanding general patterns and tendencies. The model might still be useful for identifying which situations are typically worse than others, even if it cannot provide precise predictions for individual cases.

2. Model Misspecification Error

This error occurs when the true function f is not contained within the candidate set H. Since we restricted ourselves to linear models, we may be missing important nonlinear relationships.

This error can be written as

$$f(x) - h^*(x)$$

Formatted: Centered

For example, the effect of time of day on parking may not be linear. Parking availability might be relatively stable during certain hours and then change sharply during peak times. A linear model would not capture this behavior well.

3. Estimation Error

Estimation error arises because we are using a finite dataset to estimate the model. Even if the correct model exists within H, we may not recover it exactly due to limited data.

This error is represented as

$$g(x) - h^*(x)$$

Formatted: Centered

In this case, estimation error may be smaller than the other two sources, especially if n is large. However, it is still present.

Combined Error

Putting everything together, the full relationship can be written as

$$y = g(x) + (g(x) - h^*(x)) + (f(x) - h^*(x)) + (t(z) - f(x))$$

Formatted: Centered

Each term corresponds to estimation error, misspecification error, and ignorance error, respectively.

Deleted: ¶

In this problem, the dominant term is almost certainly $t(z) - f(x)$, the ignorance error. The true causal drivers - such as the exact availability of spots, the behavior of other drivers, and the timing of departures - are fundamentally unobservable, and no amount of additional data or model complexity can recover them. This means that ignorance error effectively sets a floor on how well the model can ever perform.

Misspecification error is likely the second largest. The relationship between features and parking time is probably not linear. Time of day, for instance, likely has sharp effects during peak hours rather than a gradual increase and a linear model will not capture this well. This error can in principle be reduced by expanding H to include nonlinear terms or interactions, as discussed later in the paper.

Estimation error is likely the smallest of the three. Since parking events are frequent and n can be made large, the gap between the best function in H and the one we actually estimate

should be relatively small. This is the most controllable source of error and the least concerning in this context.

Deleted: In this problem, the dominant term is almost certainly t(z) − f(x), the ignorance error. The other two terms: misspecification and estimation, can in principle be reduced by choosing a better model or collecting more data. Ignorance error cannot.¶

Prediction Error Metrics

To evaluate the model, we need a way to measure how accurate its predictions are. One common metric is Root Mean Squared Error (*RMSE*), defined as

$$RMSE = \sqrt{SSE/n}$$

RMSE measures the typical size of prediction errors. It is useful because it is in the same units as the response variable, making it easy to interpret. In this context, that means that *RMSE* is expressed in minutes, so a value of 8 would mean the model is off by about 8 minutes on average. Given that most parking searches in New York City probably fall somewhere between 0 and 20 minutes, an *RMSE* of that size would represent a substantial portion of the total range and would likely make the model too unreliable to be useful in practice.

Formatted: Font: Italic, Complex Script Font: Italic

Formatted: Font: Italic, Complex Script Font: Italic

Another common metric is R^2, which measures the proportion of variance explained by the model. An R^2 of 1 would mean the model explains all variation in parking time, while an R^2 of 0 would mean it does no better than the null model. In this context, R^2 is less informative than RMSE because it does not tell us directly how far off predictions are in minutes. A model could have a moderate R^2 while still producing errors large enough to be useless to a driver trying to decide whether to search for street parking or use a garage.

Deleted: However, in this context, *RMSE* is more informative because it directly tells us how far off the predictions are in minutes.

In this model, the relevant *RMSE* is the out-of-sample version computed through K-fold cross-validation, not the in-sample version, for reasons discussed in the validation section.

Threshold for Usefulness

For the model to be useful, its prediction error must be small enough to provide meaningful guidance.

In this case, a reasonable threshold for usefulness might be around 2 to 3 minutes. If the model can consistently predict parking time within this range, it would be helpful for drivers. However, if the error is closer to 10 minutes, the predictions would be too unreliable to be useful in practice.

Given the nature of the problem, it is likely that the model's *RMSE* will exceed this threshold, limiting its usefulness.

Interpolation vs Extrapolation

Interpolation refers to making predictions within the range of the training data, while extrapolation refers to making predictions outside that range.

In this case, most predictions would involve interpolation. For example, predicting parking time in Manhattan at 6 PM based on past observations of similar conditions. As long as the model has seen enough examples of that combination of features, it is operating within the range of the training data. This is the intended use case, a driver entering a neighborhood under conditions the model has encountered before.

Extrapolation would occur if the model were used in entirely new settings. The most obvious example would be applying the model to a different city, where parking dynamics, street layouts, and regulations are different enough that the learned relationships may not hold. Unusual conditions within New York City could also bring the model into extrapolation territory.

Deleted: , such as a different city or under unusual conditions. These predictions would be less reliable.

For instance, a major event like a parade or weather (e.g. a blizzard) severe enough to change normal parking patterns in ways the training data did not capture.

One thing I realized while thinking about interpolation versus extrapolation is that even interpolation in this problem is not as straightforward as it sounds. Normally, interpolation is considered safer because we are making predictions within the range of observed data. However, in this case, even if two situations look very similar in terms of features, the outcomes can still be very different due to small, unobserved changes in the environment.

For example, two observations might both be in the same neighborhood at the same time of day with similar traffic conditions, but one might have several parking spots opening up while the other has none. From the model's perspective, these situations look almost identical, but the outcomes are very different. This is a direct consequence of the large ignorance error discussed earlier. The features do not capture enough of the true causal drivers to distinguish between these two situations. This makes interpolation less reliable than it would be in more stable systems and means that even predictions made well within the training data range should be interpreted with caution.

Deleted: .

Stationarity

Stationarity is the assumption that the relationship between features and the response remains stable over time.

In this context, stationarity may not fully hold. Changes in traffic patterns, construction, or parking policies could alter the relationship between features and parking time. For example, if a new building opens in a neighborhood and brings significantly more residents or visitors, the

baseline difficulty of finding parking in that area could shift permanently. Similarly, changes to street cleaning schedules or the introduction of new parking restrictions would directly affect the relationship between the restrictions feature and parking time. In these cases, a model trained on older data would no longer reflect current conditions, and its predictions would become increasingly unreliable over time.

Seasonal changes are another source of non-stationarity worth considering. Parking patterns in the summer may differ from those in the winter. A model trained primarily on data from one season might not generalize well to another.

However, over short periods, it is reasonable to assume approximate stationarity. If the model is retrained regularly on recent data, the effects of gradual shifts in parking patterns can be partially accounted for. This suggests that any practical deployment of this model would need to include a mechanism for periodic retraining, rather than being fit once and left unchanged.

Underfitting and Overfitting

Underfitting occurs when the model is too simple to capture the underlying structure of the data. Since we are using a linear model, underfitting is a real concern. The relationship between parking time and features like time of day or location is unlikely to be purely linear. There are probably sharp changes at certain hours or in certain neighborhoods that a linear model will smooth over. The result would be a model that performs poorly even on the training data, consistently missing patterns that a more flexible model could capture. In this context, underfitting would show up as a high in-sample *RMSE* and a low R^2, indicating that the model is not even learning the general structure of the training data well.

Formatted: Font: Italic, Complex Script Font: Italic

Formatted: Font: Not Italic, Complex Script Font: Not Italic

Formatted: Font: Not Italic

Formatted: Font: Not Italic

Overfitting occurs when the model is too complex and captures noise instead of true patterns. While our current model is relatively simple, overfitting could become an issue if we add too many features or transformations. This is a particular concern in this problem because the data is noisy. Driver-reported timestamps and subjective features like perceived traffic level introduce variability that has nothing to do with the true underlying phenomenon. A model with too many parameters could learn these inconsistencies rather than the actual signal, performing well on training data but poorly on new observations. This would show up as a large gap between in-sample and out-of-sample RMSE, which is one reason why K-fold cross-validation is especially important here.

The more pressing concern for this model is probably underfitting. The features aren't the strongest proxies for the true causal drivers, and a linear model is unlikely to extract much from them. Adding complexity might help at the margins, but it cannot compensate for the limitation that the most important variables are unobservable.

Validation

Moved (insertion) [1]

Once a model is selected, it must be validated to estimate its predictive performance. Validation involves separating the data into training and testing sets.

The model is trained on the training data and then evaluated on the test data. This provides an estimate of out-of-sample performance, which is more realistic than in-sample performance.

When a model is evaluated on the same data it was trained on, the resulting error metrics are called in-sample metrics. These include in-sample *SSE*, in-sample *MSE*, and in-sample

RMSE. While these are useful for understanding how well the model fits the training data, they are not trustworthy measures of how the model will perform on new observations.

This is especially relevant for the parking model. Because the features in this model are weak proxies for the true causal drivers, the model may appear to fit the training data reasonably well while still failing badly on new data. The in-sample *RMSE* could be misleadingly low simply because the model has learned the specific noise patterns in the training set rather than the underlying structure of the phenomenon. This is precisely the overfitting problem described earlier, and it means that in-sample error alone cannot be used to evaluate whether the model meets the usefulness threshold.

Out-of-sample *RMSE*, computed on held-out data the model has never seen, is the honest measure. In this problem, the gap between in-sample and out-of-sample *RMSE* is likely to be significant. The self-reported nature of the data introduces inconsistencies that the model may partially memorize during training but will not generalize from. A large gap between the two would itself be evidence of the model's limitations, independent of whether either number crosses the 2 to 3 minute threshold. For this reason, out-of-sample *RMSE* computed through K-fold cross-validation is the metric that should be used to draw any final conclusions about the model's usefulness.

One limitation of the simple train-test split described above is that the out-of-sample error estimate depends heavily on which observations happen to fall in the test set. If the test set is small, the estimate may be unreliable. To address this, a more robust approach is K-fold cross-validation.

In K-fold cross-validation, the dataset *D* is randomly divided into K equally sized subsets, called folds. The model is then trained and tested K times. In each iteration, one fold is

held out as the test set and the remaining K-1 folds are used for training. This produces K separate out-of-sample RMSE estimates, which we can denote $se_1, se_2, ..., se_k$. The average of these gives a more stable estimate of how the model performs on unseen data.

Typical choices are K = 5 or K = 10. A special case is K = n, known as leave-one-out cross-validation, where each observation is held out individually. While this is the most thorough approach, it is computationally expensive and K = 5 or K = 10 is generally preferred in practice.

For this model, K-fold cross-validation is particularly important because of the noise inherent in driver-reported data. A single train-test split might produce an optimistic or pessimistic estimate depending on which observations end up in the test set. By averaging across K splits, we get a more honest picture of model performance.

The K-fold procedure also allows us to construct a confidence interval for out-of-sample RMSE. If we let $\bar{s}_e$ denote the mean of the K-fold *RMSE* values and ss_e denote their standard deviation, a conservative estimate of prediction performance is:

$$CI = [\bar{s}_e \pm 2ss_e]$$

This interval gives a range of plausible values for how well the model will predict on future observations. Given the large ignorance error in this problem, it is likely that this interval will sit well above the 2 to 3 minute threshold defined earlier. If that is the case, the K-fold results would provide formal, data-driven confirmation of the central claim of this paper: that parking search time in New York City is difficult to model very well.

One additional concern worth noting is that driver-reported timestamps introduce a form of measurement noise that is hard to quantify. Different drivers may define the start of their search differently, and some may round their times. This does not invalidate the approach, but it

Deleted:

Deleted: cannot be

Deleted: ed

does mean that out-of-sample error estimates should be interpreted with some caution - the noise in y itself sets a floor on how low *RMSE* can realistically go.

Model Selection Problem

The model selection problem arises when choosing the best model from many possible candidates. Different choices of features, transformations, and algorithms can lead to different models, each with different performance.

In this case, choosing the appropriate candidate set H and selecting features is a key part of the modeling process. For example, we could restrict H to simple linear functions of the original features, or expand it to include polynomial terms, interaction effects, or entirely different algorithm classes. Each of these choices produces a different model with different tradeoffs between flexibility and interpretability.

Formatted: Font: Italic, Complex Script Font: Italic

The challenge is that there is no way to know in advance which candidate set will perform best. A linear model is easy to interpret and less prone to overfitting, but it may underfit if the true relationships are nonlinear. A more flexible model might capture additional structure in the data, but risks overfitting to noise, which is a real concern given how noisy the self-reported data in this problem is likely to be.

This is why model selection cannot be done by looking at in-sample performance alone. A more complex model will almost always fit the training data better, but that does not mean it

will perform better on new observations. Out-of-sample *RMSE*, computed through K-fold cross-validation, is the more honest criterion for comparing candidate models here.

It is also worth noting that even the best model found through this process will still be limited by ignorance error. Model selection can help with misspecification and estimation, but it cannot recover the unobservable causal drivers.

Formatted: Font: Italic, Complex Script Font: Italic

Forward Stepwise Regression

One systematic way to select a good model is through forward stepwise regression. This procedure begins with a simple model, typically the null model g_0, and then gradually adds features or transformations that improve predictive performance.

At each step, the algorithm considers adding a new feature or transformation and evaluates whether it reduces prediction error. This process continues until no further improvement is observed.

In this context, we could begin with:

Formatted: Indent: First line: 0.5"

$g_0(x) = \bar{y}$

and then add features such as time of day, location, and interactions one at a time. The final model is selected based on its performance on validation data. This includes raw features $x_1, \ldots, x_7$, but also nonlinear transformations and interaction terms, as well as interactions between transformed features. The full candidate pool can become very large with 7 raw features, their transformations, and pairwise interactions.

At each step of the procedure, the term that most reduces out-of-sample *RMSE* is added to the model. The final model is selected by stopping when adding further terms no longer

Formatted: Font: Italic, Complex Script Font: Italic

meaningfully improves performance on validation data, which in this context means K-fold cross-validation *RMSE*. This prevents the procedure from adding terms that improve in-sample fit but hurt generalization.

Sample Size and Feasibility of Stepwise Selection

Forward stepwise regression requires a sufficiently large dataset to avoid overfitting. If the number of observations n is too small relative to the number of features and transformations, the model may become unstable – meaning small changes in the data could lead to very different models being selected.

If we include raw features, polynomial transformations, and pairwise interactions, the total number of candidate terms could easily reach into the hundreds. We want n to be meaningfully larger than the number of terms we are considering at any given step of the procedure. If n is too close to the number of terms, the model risks overfitting to the training data and the stepwise selection process becomes unreliable.

In this case, since parking events occur frequently, it is reasonable to assume that a large dataset can be collected. Therefore, it is feasible to apply forward stepwise regression without severe limitations.

Feature Transformations and Interactions

Formatted: Font: Italic, Complex Script Font: Italic

Formatted: Font: Italic, Complex Script Font: Italic

Formatted: Font: Italic, Complex Script Font: Italic

Deleted: .

Deleted: ¶

Formatted: Indent: First line: 0"

So far, the model has assumed a simple linear relationship between features and parking time. However, it is likely that some of the relationships are not purely linear. For example, the effect of time of day on parking availability may increase sharply during peak hours rather than gradually. Similarly, the impact of location may depend heavily on time, meaning that certain neighborhoods are only difficult at specific times.

To account for this, it may be beneficial to transform some of the features or include interaction terms. For example, we could include:

x_2^2 *(time of day squared)*

to capture nonlinear effects. Parking difficulty likely does not increase at a constant rate throughout the day - it probably spikes during morning and evening rush hours and is relatively stable in between. A squared term allows the model to represent this kind of curved relationship, which a purely linear term cannot.

We could also include interaction terms such as:

$x_1 \cdot x_2$ *(location × time of day)*

The effect of time of day on parking difficulty is likely not the same across all neighborhoods. A residential area in Queens might see peak difficulty in the evening when residents return home, while a commercial area in Midtown might be hardest to park in during business hours. An interaction term between location and time of day allows the model to capture neighborhood-specific patterns like these rather than assuming the same effects everywhere.

These transformations expand the feature space and allow the model to capture more complex relationships.

Deleted: ,

Deleted: or

Deleted: to model how the effect of one feature depends on another.¶

Other Algorithm Choices

While *OLS* is a natural starting point, other algorithms could also be considered in this context. K-Nearest Neighbors, for example, could predict search time based on similarity to past observations with matching conditions - finding the *k* most similar historical parking searches and averaging their outcomes. What makes KNN a good option here is that it makes no assumptions about the relationship between features and parking time. Unlike *OLS,* which assumes a linear relationship, *KNN* simply looks for past situations that resemble the current one and uses their outcomes directly. This is good for this problem, where relationships are likely nonlinear and complex. It also makes sense intuitively; if a driver is searching for parking, averaging previous outcomes under the most similar situation is a reasonable way to form a prediction.

However, even the most flexible algorithm cannot compensate for the fundamental absence of the true causal drivers - no model has access to the real-time availability of parking spaces or the behavior of other drivers in the moment. KNN can only find situations that look similar, but two observations that appear to have identical features can still have different outcomes due to factors that the model cannot see. This means that ignorance error is still the biggest limitation on this model regardless of the algorithm choice.

Using the Model for Prediction

Once validated, the model can be used to make predictions in real-world situations. Given a set of features describing current conditions, the model outputs a predicted parking time.

For example, a driver entering a neighborhood at 7 PM on a weekday with high traffic could receive a prediction such as:

$\hat{y} = 12$ *minutes*

Formatted: Font: Italic, Complex Script Font: Italic

Formatted: Font: Not Italic, Complex Script Font: Not Italic

Formatted: Font: Italic, Complex Script Font: Italic

Formatted: None, Indent: First line: 0.5"

Deleted: ¶
¶
¶

Moved up [1]: Validation¶
Once a model is selected, it must be validated to estimate its predictive performance. Validation involves separating the data into training and testing sets.¶
The model is trained on the training data and then evaluated on the test data. This provides an estimate of out-of-sample performance, which is more realistic than in-sample performance.¶
When a model is evaluated on the same data it was trained on, the resulting error metrics are called in-sample metrics. These include in-sample *SSE*, in-sample *MSE*, and in-sample *RMSE*. While these are useful for understanding how well the model fits the training data, they are not trustworthy measures of how the model will perform on new observations.¶
This is especially relevant for the parking model. Because the features in this model are weak proxies for the true causal drivers, the model may appear to fit the training data reasonably well while still failing badly on new data. The in-sample *RMSE* could be misleadingly low simply because the model has learned the specific noise patterns in the training set rather than the underlying structure of the phenomenon. This is precisely the overfitting problem described earlier, and it means that in-sample error alone cannot be used to evaluate whether the model meets the usefulness threshold.¶
Out-of-sample *RMSE*, computed on held-out data the model has never seen, is the honest measure. In this problem, the gap between in-sample and out-of-sample *RMSE* is likely to be significant. The self-reported nature of the data introduces inconsistencies that the model may partially memorize during training but will not generalize from. A large gap between the two would itself be evidence of the model's limitations, independent of whether either number crosses the 2 to 3 minute threshold. For this reason, out-of-sample *RMSE* computed through K-fold cross-validation is the metric that should be used to draw any final conclusions about the model's usefulness.¶
One limitation of the simple train-test split described above is that the out-of-sample error estimate depends heavily on which observations happen to fall in the test set. If the test set is small, the estimate may be unreliable. To address this, a more robust approach is K-fold cross-validation.¶
In K-fold cross-validation, the dataset *D* is randomly divided into K equally sized subsets, called folds. The model is then trained and tested K times. In each iteration, one fold is held out as the test set and the remaining K-1 folds are used for training. This produces K separate out-of-sample RMSE estimates, which we can denote $se_1, se_2, \ldots, se_k$. The average of these gives a more stable estimate of how the model performs on unseen data.¶
Typical choices are K = 5 or K = 10. A special case is K = n, known as leave-one-out cross-validation, where each observation is held out individually. While this is the most

This information could help the driver decide whether to continue searching for street parking or choose an alternative. If the predicted time is low, searching for street parking may be worth it. If the predicted time is high, a driver might decide that paying for a garage is the better option.

Beyond individual decisions, the model could also be integrated into navigation systems. A navigation app could incorporate the predicted parking time into the total estimated trip time, giving a more realistic idea of when the driver will arrive.

The model could also be used more broadly to identify patterns across neighborhoods and times of day. Even if individual predictions are imprecise, the model might be able to reliably indicate that certain areas are consistently harder to park in at certain times. In that case, the model might suggest that parking a few blocks further away and walking to the destination would ultimately be faster than continuing to search for a spot closer in.

Deleted: ¶ ... [1]

Final Discussion on Usefulness

Even with improvements such as feature transformations and more advanced algorithms, the model will still face limitations due to the unpredictable nature of parking availability.

The threshold for usefulness was previously defined as an error of approximately 2 to 3 minutes. Given the large role of randomness in this problem, it is unlikely that the model will consistently achieve this level of accuracy.

While the model may still provide general estimates, such as identifying areas where parking is typically difficult, it will likely struggle to make precise predictions for individual parking attempts.

Conclusion

The goal of this paper was to determine whether the time required to find street parking in New York City can be modeled accurately. While it is possible to construct a model using observable features such as location, time of day, and traffic conditions, the results are fundamentally limited by the presence of unpredictable factors. The most direct way to confirm this would be through K-fold cross-validation, where the resulting confidence interval for out-of-sample *RMSE* would be expected to sit well above the threshold for usefulness.

The largest source of error in this model is error due to ignorance, which arises from the inability to observe key causal drivers such as the exact availability of parking spaces and the behavior of other drivers. Model misspecification and estimation error also contribute, but to a lesser extent.

A key takeaway from working through this model is how much of the outcome depends on factors that are not observable ahead of time. At first, it seems like adding more features or collecting more data would significantly improve the model. However, once the role of the true causal drivers is considered, it becomes clear that some of the most important variables are simply not accessible. Because of this, the limitations of the model are not just due to a lack of data, but are built into the nature of the problem itself.

This also changes what it means for the model to be useful. In many cases, a good model is expected to make highly accurate predictions. In this context, that standard is probably unrealistic. A more reasonable goal is for the model to capture general patterns, such as which times or locations tend to have worse parking conditions. Even if it cannot predict the exact time for a specific situation, it can still provide useful insight at a broader level.

From a practical perspective, this means that the model is better suited for giving general guidance rather than precise predictions. For example, it might be able to indicate that parking will likely take longer than usual in a certain area, but it cannot reliably predict whether a driver will find a spot in two minutes or fifteen. That level of uncertainty is unavoidable given how dynamic the system is.

Based on the analysis presented in this paper, the time required to find street parking in New York City is difficult to model accurately when it comes to making precise individual predictions. The structural limitations imposed by unobservable causal drivers make consistently accurate predictions difficult to achieve in practice. However, that does not mean the model is completely useless. Statistical and machine learning models can still provide useful approximations and help identify broader patterns in parking behavior. Even if the model cannot reliably predict whether a driver will find parking in two minutes or fifteen, it may still be able to indicate which neighborhoods, times of day, or conditions are generally associated with more difficult parking. The randomness and instability inherent in the parking process place a limit on predictive performance, but the modeling process can still provide meaningful practical value despite those limitations.

Deleted: This supports the claim made in the title: the time required to find street parking in New York City cannot be modeled very well. While models can still provide useful approximations and general insights, they cannot fully capture the complexity and randomness of the phenomenon.

Works Cited

Google. (2017, January 26). *Know before you go: Parking difficulty on Google Maps.* Google Blog. https://blog.google/products/maps/know-you-go-parking-difficulty-google-maps/

New York City Department of City Planning. (2009). *Residential parking study: Automobile ownership rates and off-street parking requirements in portions of New York City.* https://www.nyc.gov/assets/planning/download/pdf/plans/transportation/residential_parking.pdf

New York City Department of City Planning. (2011). *Manhattan Core public parking study.* https://www.nyc.gov/assets/planning/download/pdf/plans/manhattan-core-public-parking/mncore_study.pdf

San Francisco Municipal Transportation Agency. (2013). *SFpark pilot program.* https://www.sfmta.com/projects/sfpark-pilot-program

Page 28: [1] Deleted **Isa Nawaz** **5/6/26 1:45:00 PM**